Advanced Programming Methods

Lecture 2 – Object-Oriented Design in Java

```
sealed interface Forma permits Cerc, Patrat { }
```

Lecture overview

01 Composition and inheritance

02 Polymorphism and abstract classes

03 Interfaces (incl. default methods)

04 Casting and instanceof

05 The class Object: equals, hashCode, toString

06 Nested and anonymous classes

07 Enums, records, sealed types, pattern matching

08 Packages, access modifiers, modules

01

Code reuse: composition and inheritance

Two ways to reuse code

`has-a`

Composition

The new class contains fields that are objects of existing classes.

Persoana has an Adresa

`is-a`

Inheritance

The new class extends an existing class: it adds new fields and methods to those it inherits.

Angajat is a Persoana

Composition

- The new class contains fields that are objects of existing classes

Adresa.java

```
class Adresa {
    private String nr, strada,
                   localitate, tara;
    private long codPostal;
    //...
}
```

Persoana.java

```
class Persoana {
    private String nume;
    private Adresa adresa;
    private String cnp;
    //...
}
```

Scrisoare.java

```
class Scrisoare {
    private String destinatar;
    private Adresa adresaDestinatar;
    private String expeditor;
    private Adresa adresaExpeditor;
    //...
}
```

Inheritance

child parent

```
class NewClass extends ExistingClass {
    //...
}
```

`NewClass` is the **subclass** (child class, derived class); `ExistingClass` is the **superclass** (parent class, base class).

`NewClass` inherits all members of `ExistingClass`. It may **override** some of the inherited methods and **add** new fields and methods.

Private members are inherited but not directly accessible in the subclass. => getters, setters

Inheritance – example

Punct.java

```java
public class Punct {
    private int x, y;

    public Punct(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public void muta(int dx, int dy) {
        //...
    }
    public void deseneaza() {
        //...
    }
}
```

PunctColorat.java

```java
public class PunctColorat extends Punct {
    private String culoare;

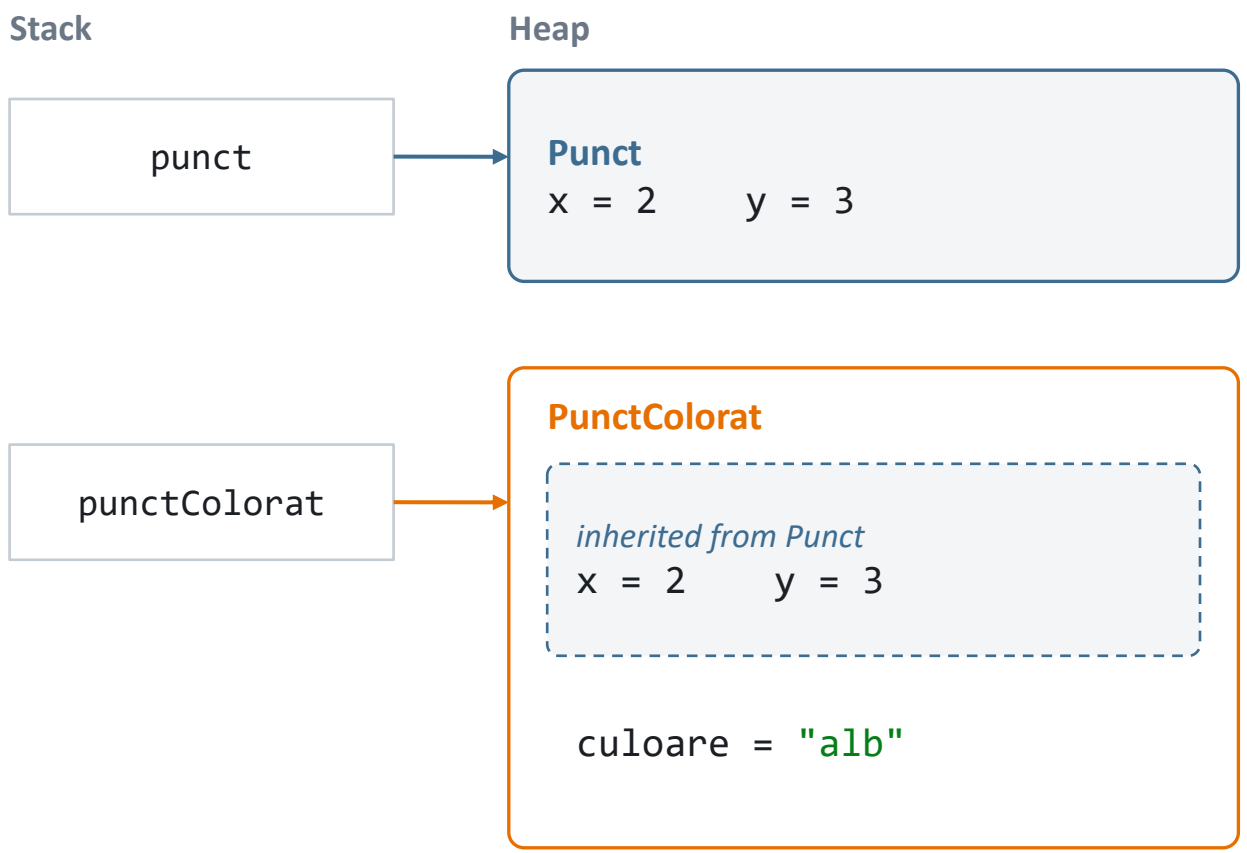
    public PunctColorat(int x, int y,
                        String culoare) { /*...*/ }

    public void deseneaza() { /*...*/ }  // overridden
    public String culoare() { /*...*/ }  // new
}
// muta(...) is inherited unchanged
```

Inheritance – notions and memory layout

- `deseneaza` is an **overridden** method
- `muta` is an **inherited** method
- `culoare` is a **newly added** method

```
Punct punct = new Punct(2, 3);
PunctColorat punctColorat =
    new PunctColorat(2, 3, "alb");
```



Method overriding

- A subclass may **override** methods of its base class
- Class `B` overrides method `mR` of class `A` if `B` defines `mR` with the **same signature** as in `A`
- For a call `a.mR()`, the method executed is chosen by the **class of the object** referred to by `a`, not by the declared type of `a`
- **Static** methods are not overridden but *hidden*: a call is resolved using the declared type

```
A a = new A();
a.mR();          // mR from A

a = new B();
a.mR();          // mR from B
```

@Override and covariant return types

- `@Override` asks the compiler to check that the method really overrides one from the base class
- The return type of an overriding method may be a **subtype** of the original return type (*covariant return type*)
  ↳ when a subclass overrides a method, it's allowed to declare a more specific return type than the one in the parent class (as long as that type is a subtype of the original)

A.java / B.java

```java
public class A {
    public void mR() { /*...*/ }
}
public class B extends A {
    @Override
    public void mR() { /*...*/ }
}
```

C.java / D.java

```java
public class SubB extends SuperA { /*...*/ }

public class C {
    public SuperA m() { /*...*/ }
}
public class D extends C {
    @Override
    public SubB m() { /*...*/ }   // covariant
}
```

Method overloading in subclasses

- A subclass may **overload** a method inherited from the base class
- A subclass object can call **all** overloaded versions, including those from the superclass

A.java / B.java

```
public class A {
    public void f(int h) { /*...*/ }
    public void f(int i, char c) { /*...*/ }
}
public class B extends A {
    public void f(String s, int i) { /*...*/ }
}
```

usage

```
B b = new B();
b.f(23);          // from A
b.f(2, 'c');      // from A
b.f("mere", 5); // from B
```

Calling the superclass constructor: super(...)

- A subclass constructor can call a base class constructor using `super(...)`
- Traditionally the **first statement**; since Java 25 it may be preceded by code that does not use `this` (e.g. argument validation)
- If it is missing, the compiler inserts `super()` – which must exist

Persoana.java

```
public class Persoana {
    private String nume;
    private int varsta;
    public Persoana(String nume, int varsta) {
        this.nume = nume;
        this.varsta = varsta;
    }
}
```

Angajat.java

```
public class Angajat extends Persoana {
    private String departament;
    public Angajat(String nume, int varsta, String departament) {
        super(nume, varsta);
        this.departament = departament;
    }
}
```

The keyword super

- Calls a constructor of the base class: `super(...)`
- Refers to a base-class member hidden in the subclass: `super.ac`
- Calls the overridden base-class method from the overriding method: `super.deseneaza()`

field hiding

```
public class A {
    protected int ac = 3;
}
public class B extends A {
    protected int ac = 3;      // hides A.ac
    public void f() {
        ac += 2;               // B.ac
        super.ac--;            // A.ac
    }
}
```

calling the overridden method

```
public class PunctColorat extends Punct {
    private String culoare;
    @Override
    public void deseneaza() {
        System.out.println(culoare);
        super.deseneaza();    // Punct's version
    }
}
```

02

Polymorphism and abstract classes

Polymorphism

- The ability of an object to behave differently depending on the context

Three kinds of polymorphism

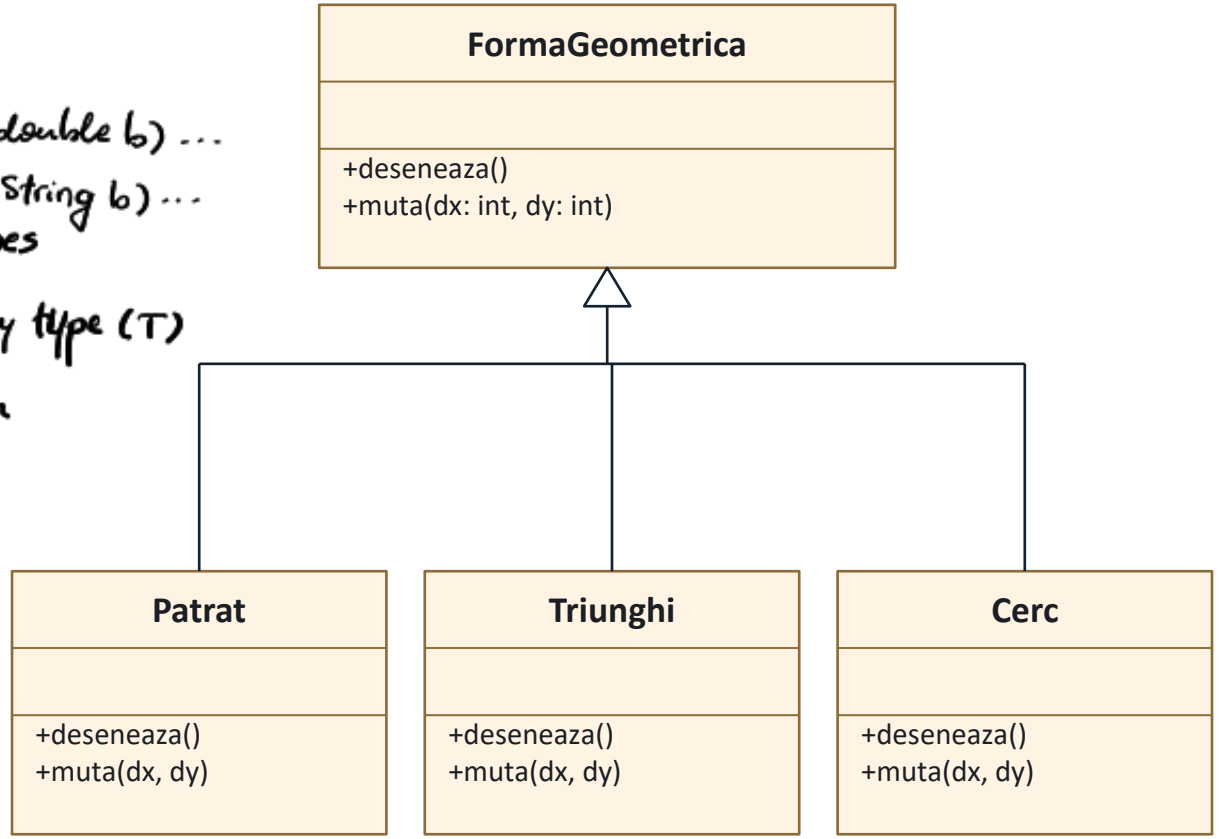
double add (double a, double b) ...
String add (String a, String b) ...

- **Ad-hoc** – method overloading → same name on ≠ types
- **Parametric** – generic types → one implem. works for any type (T)
- **Inclusion (subtype)** – inheritance → dynamic dispatch

late binding (runtime)

deciding which method version to run when calling an overridden method.

- reference type can be ≠ from obj. type
  ex: Animal a1 = new Dog();
- ~~Compile~~ - time → runtime (JVM)
  - variable's label
  - check actual object in memory



Early and late binding

- **Early binding:** the method to execute is decided at **compile time**
- **Late binding:** the method to execute is decided at **run time**, from the class of the object
- Java uses late binding for instance methods
- Exceptions: `static` and `private` methods are bound at compile time

```
void deseneaza(FormaGeometrica fg) {
    fg.deseneaza();
}
//...
FormaGeometrica fg = new Patrat();
deseneaza(fg);   // calls deseneaza from Patrat
fg = new Cerc();
deseneaza(fg);   // calls deseneaza from Cerc
```

Polymorphic collections

- One array of the base type can hold objects of **any subclass**
- Each element reacts with **its own** `muta` / `deseneaza`

```
public FormaGeometrica[] genereaza(int dim) {
    FormaGeometrica[] fg = new FormaGeometrica[dim];
    Random rand = new Random(47);
    for (int i = 0; i < dim; i++) {
        switch (rand.nextInt(3)) {
            case 0: fg[i] = new Cerc(); break;
            case 1: fg[i] = new Patrat(); break;
            case 2: fg[i] = new Triunghi(); break;
        }
    }
    return fg;
}

public void muta(FormaGeometrica[] fg) {
    for (FormaGeometrica f : fg)
        f.muta(3, 3);
}
```

Abstract classes

- An **abstract method** is declared but not implemented; it is marked `abstract`
- An **abstract class** may contain abstract methods, fields, constructors and implemented methods

syntax

```
[access] abstract ReturnType name([params]);

[public] abstract class ClassName {
    [fields]
    [abstract methods declaration]
    [methods declaration and implementation]
}
```

Polinom.java

```
public abstract class Polinom {
    //...
    public abstract void aduna(Polinom p);
}
```

Abstract classes – rules

- An abstract class **cannot be instantiated**: `new Polinom()` is a compile error
- A class with at least one abstract method **must** be declared abstract
- A class can be abstract without having any abstract method
- A subclass that does not implement all inherited abstract methods must also be abstract

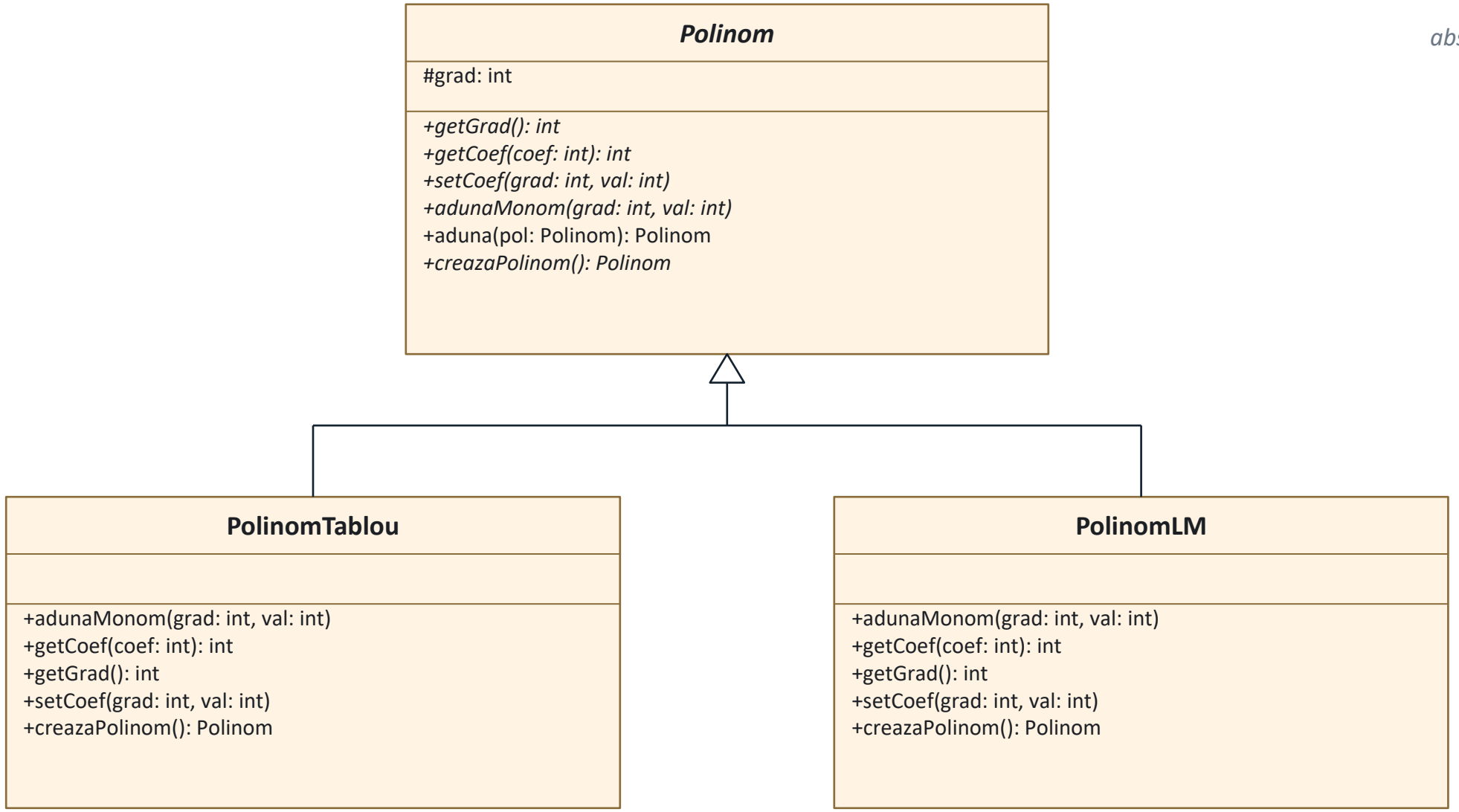
A.java

```
public abstract class A {
    public A() {}
    public abstract void f();
    public abstract void g(int i);
}
```

B.java

```
public abstract class B extends A {
    private int i = 0;        // f() is still abstract
    public void g(int i) {
        this.i += i;
    }
}
```

Abstract class example: Polinom



abstract = italic

03

Interfaces

Java interfaces

- Declared with the keyword `interface`; an interface is a **type** with no state
- No constructors, no instance fields
- Abstract methods are implicitly `public abstract`
- Fields are implicitly `public static final` (constants)
- Since Java 8: `default` and `static` methods with a body; since Java 9: `private` methods
- An interface may be empty (a *marker interface*, e.g. `Serializable`)

syntax

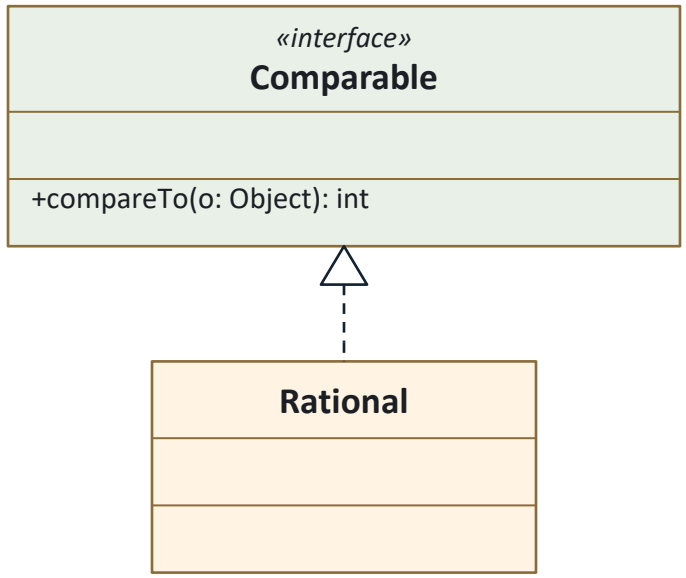
```
public interface InterfaceName {
    [constants]
    [method declarations]
}
```

Shape.java

```
public interface Shape {
    double PI2 = 2 * Math.PI;   // constant
    double area();              // abstract
    default String describe() { // Java 8+
        return "area = " + area();
    }
}
```

Implementing an interface

- A class implements an interface with `implements`
- It must implement **all** abstract methods of the interface – or be declared `abstract`



syntax

```
[public] class ClassName implements InterfaceName {
    [interface method implementations]
    // other definitions
}
```

Rational.java

```
public interface Comparable {
    int compareTo(Object o);
}
public class Rational implements Comparable {
    private int numarator, numitor;
    //...
    public int compareTo(Object o) { /*...*/ }
}
```

Extending interfaces

- An interface can extend **one or more** interfaces
- This is Java's form of **multiple inheritance** – of types, not of state

syntax

```
[public] interface Name extends I1[, I2, ...] {
    [new method declarations]
}
```

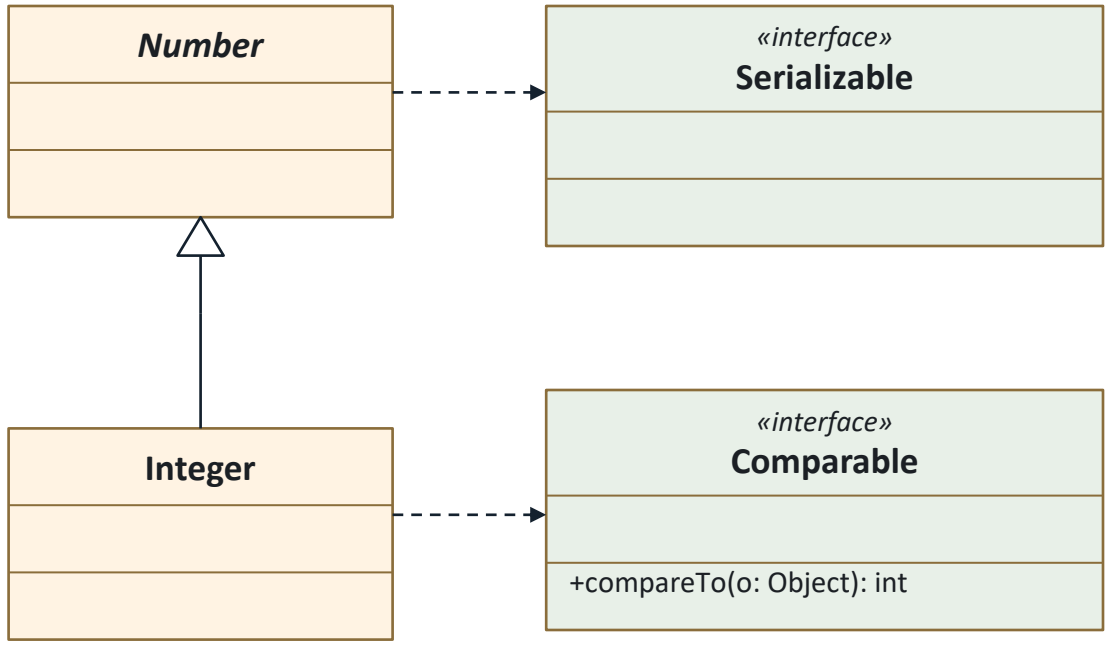
example

```
public interface A { int f(); }
public interface B { double h(int i); }

public interface C extends A, B {
    boolean g(String s);
}   // C has f, h and g
```

Inheritance and interfaces together

- A class can extend **only one** class, but can implement **any number** of interfaces



syntax

```
[public] class Name extends SuperClass
        implements I1, I2, ..., In {
    //...
}
```

simplified from java.lang

```
public class Integer extends Number
        implements Comparable {
    public int compareTo(Object o) { /*...*/ }
}
// Number implements Serializable
```

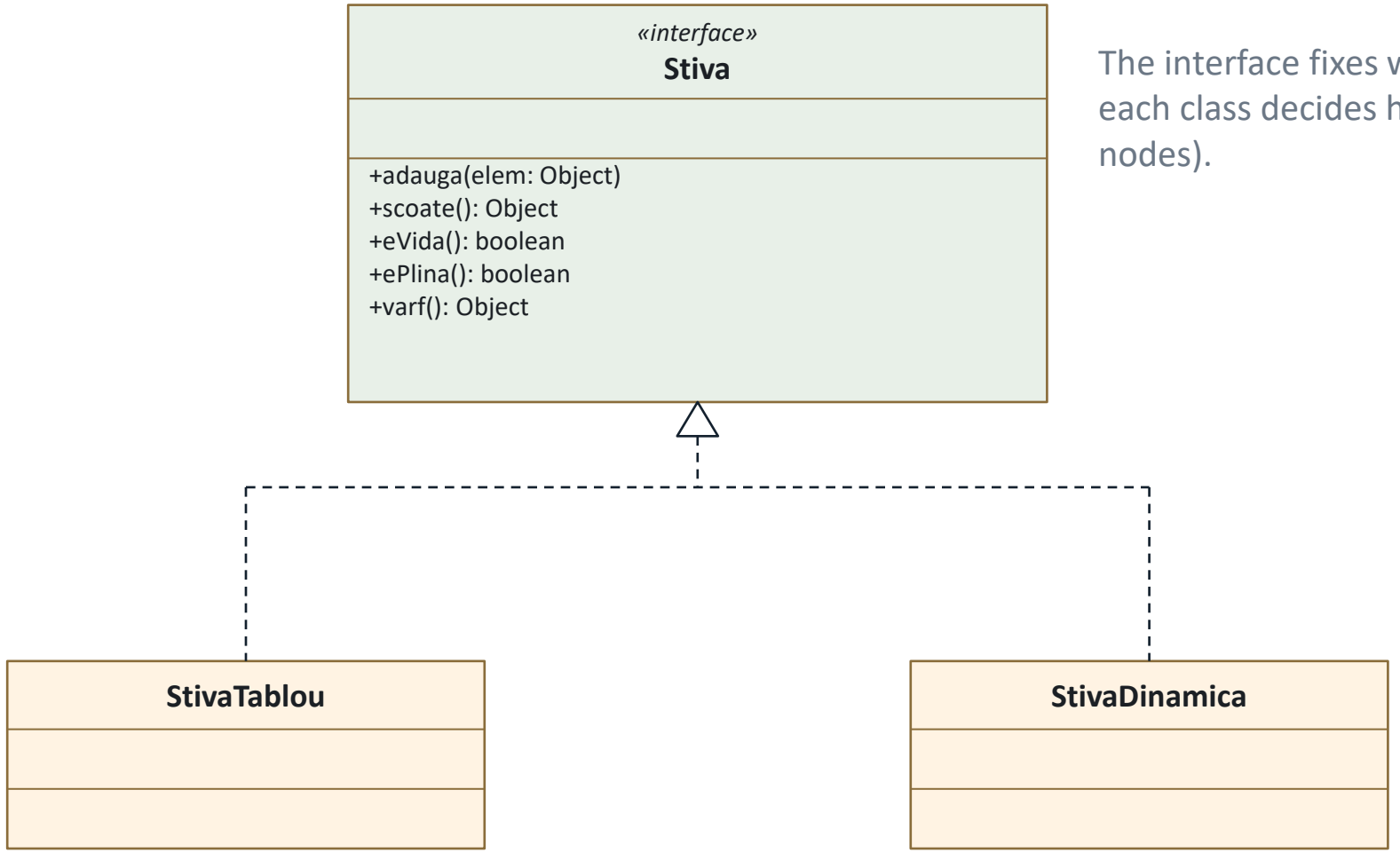
Variables of interface type

- An interface is a **reference type**: variables of interface type can be declared
- They can refer to objects of **any class that implements** the interface
- Through such a variable, **only the interface methods** can be called

```
public interface Comparable { /*...*/ }
public class Rational implements Comparable { /*...*/ }

Rational r = new Rational();
Comparable c = r;
Comparable cr = new Rational(2, 3);
cr.compareTo(c);
c.aduna(cr);    // ERROR: not in Comparable
```

Interface example: Stiva



The interface fixes what a stack can do; each class decides how (array vs. linked nodes).

Interfaces before and after Java 8

Before Java 8

- An interface defines a **contract**
- Only abstract methods and constants (`public static final`)
- Every implementing class must define **all** methods

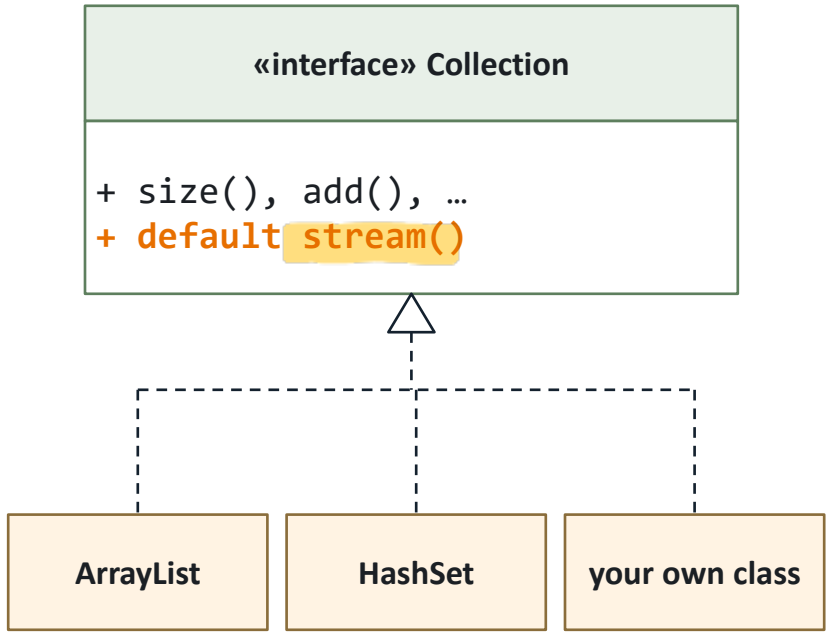
Since Java 8 (and 9)

- `default` methods: have a body; implementing classes **may override** them
- `static` methods: have a body; **cannot be overridden**; called as `Interface.method()`
- Java 9: `private` methods, to share code between default methods
- Classes must still define all **abstract** methods

Why default methods?

- If the designers of a published interface add a new method, **every** implementing class stops compiling – each must now define it
- Default and static methods let an interface **evolve** without breaking existing implementations
- The JDK itself relies on this: Java 8 added `Collection.stream()` and `List.sort(...)` as default methods

Stream = flux de data



existing implementations keep compiling

Example on the next slides adapted from examples.javacodegeeks.com (Interface vs Abstract Class)

Example: an interface with default and static methods

- `printWelcomeMessage` – **static**: cannot be overridden; `makeACall` – **default**: may be overridden; `capturePhoto` – **abstract**: must be defined

MobileInterface.java

```
public interface MobileInterface {
    // static: called as MobileInterface.printWelcomeMessage()
    static void printWelcomeMessage() {
        System.out.println("***STATIC METHOD*** Welcome!!");
    }
    // default: implementing classes inherit or override it
    default void makeACall(long number) {
        System.out.println("***DEFAULT METHOD*** Calling ... " + number);
    }
    // abstract: every implementing class must define it
    void capturePhoto();
}
```

Implementing classes

BasicPhone.java

```java
// defines only the abstract method;
// makeACall() uses the default version
class BasicPhone implements MobileInterface {
    @Override
    public void capturePhoto() {
        System.out.println(
            "***BASIC PHONE*** Cannot capture photo");
    }
}
```

SmartPhone.java

```java
// overrides the default method too
class SmartPhone implements MobileInterface {
    @Override
    public void makeACall(long number) {
        System.out.println("***SMART PHONE*** "
            + "Voice and video call to " + number);
    }
    @Override
    public void capturePhoto() {
        System.out.println(
            "***SMART PHONE*** Can capture photo");
    }
}
```



Overriding methods must be `public`: interface methods are implicitly public, and access cannot be reduced.

Using the interface

MobileInterfaceDemo.java

```java
public class MobileInterfaceDemo {
    public static void main(String[] args) {
        MobileInterface basicPhone = new BasicPhone();
        MobileInterface smartPhone = new SmartPhone();

        MobileInterface.printWelcomeMessage(); // static

        basicPhone.makeACall(1234567890L);   // default
        basicPhone.capturePhoto();           // BasicPhone

        smartPhone.makeACall(1234567890L);   // overridden
        smartPhone.capturePhoto();           // SmartPhone
    }
}
```

output

```
***STATIC METHOD*** Welcome!!
***DEFAULT METHOD*** Calling ... 1234567890
***BASIC PHONE*** Cannot capture photo
***SMART PHONE*** Voice and video
    call to 1234567890
***SMART PHONE*** Can capture photo
```

Abstract class vs. interface

	Abstract class	Interface
Methods	public, protected, package-private, private; abstract or implemented	public abstract by default; default and static (Java 8+), private (Java 9+)
Fields	any fields (instance and static)	only public static final constants
Constructors	yes (called via super(...))	no
Inheritance	a class extends one class	a class implements many interfaces
Abstract methods	may have none	may have none (marker interface)
Instances	cannot be instantiated	cannot be instantiated

04

Casting and instanceof

Upcasting and downcasting

- **Upcasting** (subclass → superclass) is implicit and always safe
- **Downcasting** (superclass → subclass) needs an explicit cast → TO BE AVOIDED
- The compiler accepts the cast; the **JVM checks it at run time** and throws `ClassCastException` if the object is not of that type

```
class Base { /*...*/ }
class SubBase extends Base { /*...*/ }

Base objB = new Base();
SubBase objSB1 = new SubBase();
SubBase objSB2;

objB = objSB1;              // OK: upcasting
objSB2 = objB;              // compile error
objSB2 = (SubBase) objB;    // OK: objB refers to a SubBase

objSB2 = (SubBase) new Base();
                // compiles, but ClassCastException at run time
```

The instanceof operator

- Tests whether an object is an instance of a type:
  - the class itself, a subclass, or a class that implements an interface
- `null instanceof T` is always `false` – `null` is not an instance of anything
- Used before a downcast to avoid `ClassCastException`

```
interface MyInterface {}
class Parent {}
class Child extends Parent implements MyInterface {}

Parent obj1 = new Parent();
Parent obj2 = new Child();

obj1 instanceof Parent        // true
obj1 instanceof Child         // false
obj1 instanceof MyInterface   // false
obj2 instanceof Parent        // true
obj2 instanceof Child         // true
obj2 instanceof MyInterface   // true
```

instanceof in practice

classic: test, then cast

```
interface Printable {}
class A implements Printable {
    public void a() { System.out.println("a method"); }
}
class B implements Printable {
    public void b() { System.out.println("b method"); }
}
class Call {
    void invoke(Printable p) {
        if (p instanceof A) {
            A a = (A) p;        // downcasting
            a.a();
        }
        if (p instanceof B) {
            B b = (B) p;        // downcasting
            b.b();
        }
    }
}
```

Java 16+: pattern matching

```
class Call {
    void invoke(Printable p) {
        if (p instanceof A a)   // test + cast + bind
            a.a();
        else if (p instanceof B b)
            b.b();
    }
}
```

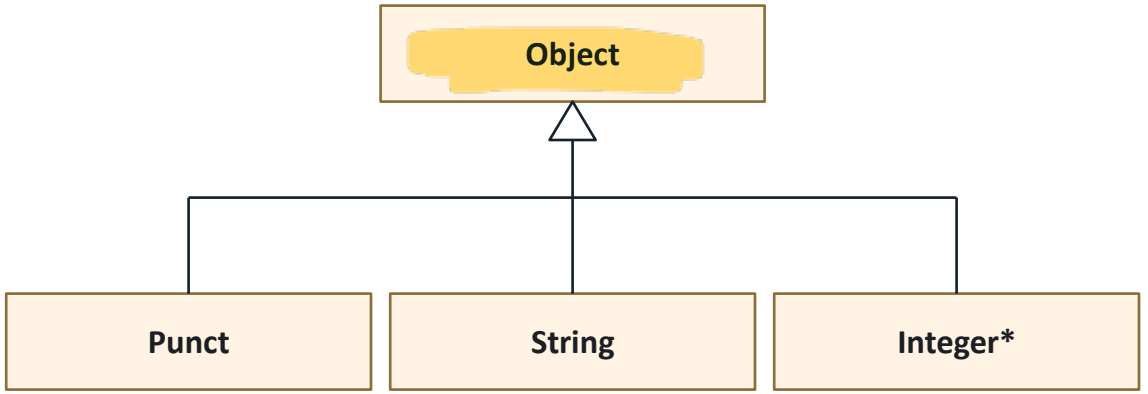
05

The class Object

Object: the root of the hierarchy

- `java.lang.Object` is the top of the Java class hierarchy
- A class without an explicit `extends` implicitly extends `Object`
- So every class inherits the methods of `Object`

```
public class Punct {                    // extends Object
    //...
}
public class PunctColorat extends Punct {
    //...
}
```



`PunctColorat → Punct → Object`

** Integer extends Number, which extends Object*

Methods of Object

- `toString()` – called automatically when a `String` is expected
- `equals(Object)` – checks equality of two objects; the default implementation compares **references** (same as `==`)
- `hashCode()` – must be consistent with `equals`




```
Punct p1 = new Punct(2, 3);
Punct p2 = new Punct(2, 3);
boolean egale = (p1 == p2);    // false (not the same obj. in memory)
egale = p1.equals(p2);         // true only if Punct
                               // overrides equals

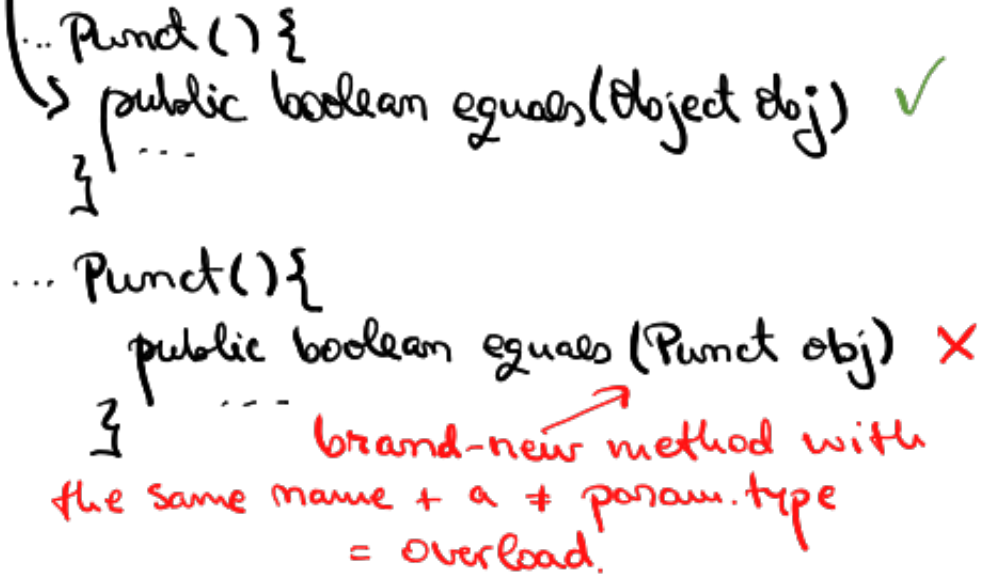
System.out.println(p1);        // calls p1.toString()
                               ↳ Punct@1a2b...
```

Object
+equals(o: Object): boolean +toString(): String +hashCode(): int +getClass(): Class<?> +notify(), notifyAll(), wait() #clone(): Object *#finalize() (deprecated)*

- equals() → compares references
- equals(Object) → (root object)
- == → primitive types (int, char, boolean) → compares values
  → checks if they point to the same object in memory

Overriding equals, hashCode and toString

- `equals` takes an `Object` parameter – otherwise it **overloads** instead of overriding
- Objects that are `equals` must have the **same** `hashCode` – required by `HashMap`, `HashSet`
- `@Override` catches signature mistakes



Punct.java

```java
public class Punct {
    private int x, y;
    public Punct(int x, int y) {
        this.x = x;
        this.y = y;
    }
    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Punct))
            return false;
        Punct p = (Punct) obj;
        return x == p.x && y == p.y;
    }
    @Override
    public int hashCode() {
        return 31 * x + y;    // or Objects.hash(x, y)
    }
    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

06

Nested and anonymous classes

Classes inside classes

Kind	Declared	Has an outer object?	Typical use
static nested	static class Nod {…} inside a class	no	helper types: list nodes, builders
inner	class Cursor {…} inside a class	yes – Outer.this	iterators, views on the outer object
local	inside a method body	yes, if in an instance method	small helpers used by one method
anonymous	new Interface() {…} in an expression	yes, if in an instance method	one-off implementations (now mostly lambdas)



Nested classes can access the **private** members of the enclosing class – and vice versa. Prefer `static` nested classes unless you really need the outer object.

Static nested and inner classes

- `Nod` is **static**: it needs no `Lista` object
- `Cursor` is an **inner** class: every cursor is tied to a `Lista` object and reads its `head`
- An inner object is created from an outer one: `l.new Cursor()`

Lista.java

```
public class Lista {
    private Nod head;

    private static class Nod {          // static nested
        int info; Nod next;
        Nod(int info, Nod next) { this.info = info; this.next = next; }
    }

    public class Cursor {               // inner
        private Nod crt = head;         // Lista.this.head
        public boolean areUrmator() { return crt != null; }
        public int urmator() {
            int v = crt.info; crt = crt.next; return v;
        }
    }
    public void adauga(int x) { head = new Nod(x, head); }
}

Lista l = new Lista();  l.adauga(3);  l.adauga(7);
Lista.Cursor c = l.new Cursor();
while (c.areUrmator()) System.out.println(c.urmator()); // 7 3
```

Anonymous classes

- Declare and instantiate a class **in one expression** – the class has no name
- Used for classes needed only once: event handlers, callbacks, comparators
- For interfaces with a single abstract method, Java 8 **lambdas** are the concise replacement (Lecture 4)

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("rulez");
    }
};
r.run();
```

Swing event handler

```
JButton b = new JButton("Test");
b.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Click detected");
    }
});
// Lecture 4: b.addActionListener(e -> System.out.println("Click"));
```

Local and anonymous classes capture variables

- A local or anonymous class can use local variables and parameters of the enclosing method
- Those variables must be **effectively final**: never reassigned after initialization
- The same rule applies to lambdas

```
int numaraPestePrag(int[] v, int prag) {
    class Contor {                  // local class
        int n;
        void verifica(int x) {
            if (x > prag) n++;      // uses the parameter prag
        }
    }
    Contor c = new Contor();
    for (int x : v) c.verifica(x);
    // prag = 0;  would make prag not effectively final
    //            -> compile error at 'x > prag'
    return c.n;
}
```

07

Enums, records and sealed types

→ when you have an unchanging collection of states.

Enums: a fixed set of named constants

↳ used to represent a fixed, predefined set of named constants ← days of the week, months.

the old way – int constants

```
public interface LuniAn {
    int IANUARIE = 1, FEBRUARIE = 2,
        /*...*/ DECEMBRIE = 12;
}
void planifica(int luna) { /*...*/ }
planifica(42);      // compiles – but is meaningless
```

with an enum

```
public enum Zi {
    LUNI, MARTI, MIERCURI, JOI,
    VINERI, SAMBATA, DUMINICA
}

Zi z = Zi.SAMBATA;
boolean weekend = switch (z) {
    case SAMBATA, DUMINICA -> true;
    default -> false;
};
Zi.valueOf("LUNI");   // Zi.LUNI
z.ordinal();          // 5
```

→ can have fields, methods etc.

i An enum is a **type**: only its constants are valid values. Enums can be used in `switch`, compared with `==`, and stored efficiently in `EnumSet` / `EnumMap`.

Enums with fields, constructors and methods

- Each constant is an **object**, created once, when the enum is loaded
- Enums can have fields, constructors (always private) and methods
- Enums can implement interfaces

Calificativ.java

```
public enum Calificativ {
    FOARTE_BINE(10), BINE(8), SUFICIENT(5), INSUFICIENT(0);

    private final int puncte;

    Calificativ(int puncte) { this.puncte = puncte; }

    public int puncte() { return puncte; }
    public boolean promovat() { return this != INSUFICIENT; }
}

for (Calificativ c : Calificativ.values())
    System.out.println(c + " " + c.puncte() + " " + c.promovat());
```

→ when you want to write a single line of code and the compiler automatically generates code.

Records: transparent data carriers (Java 16)

↳ shortcut classes build to hold immutable data

one line…

```
public record Punct(int x, int y) { }

Punct p1 = new Punct(2, 3);
Punct p2 = new Punct(2, 3);
p1.x();             // 2 – accessor
p1.equals(p2);      // true
System.out.println(p1); // Punct[x=2, y=3]
```

…the compiler generates

```
public final class Punct {
    private final int x, y;
    public Punct(int x, int y) { this.x = x; this.y = y; }
    public int x() { return x; }
    public int y() { return y; }
    public boolean equals(Object o) { /* by components */ }
    public int hashCode() { /* by components */ }
    public String toString() { /* Punct[x=2, y=3] */ }
}
```

i Compare with the hand-written `equals`/`hashCode`/`toString` of `Punct` in section 05. Records are **immutable** (final fields) and cannot extend other classes, but can implement interfaces.

Records: validation and extra methods

→ easy to add a new thing

- A **compact constructor** validates or normalizes the components
- Records may add methods and static factories
- Immutability is **shallow**: a record holding a mutable list does not protect the list

Student.java

```
public record Student(String nume, double media) {

    public Student {                          // compact constructor
        Objects.requireNonNull(nume);
        if (media < 1 || media > 10)
            throw new IllegalArgumentException("media: " + media);
    }

    public boolean integralist() { return media >= 5; }

    public static Student parse(String linie) {   // "Pop Ion|9.3"
        String[] p = linie.split("\\|");
        return new Student(p[0].strip(), Double.parseDouble(p[1]));
    }
}
```


→ when you want to do new restrictions

ex: Forma → can only be Cerc / Patrat / Dreptunghi. → harder to add a new thing

Sealed classes and interfaces (Java 17)

↳ restricts who can inherit from this class / interface

- `sealed` … `permits` fixes the **complete list** of direct subtypes
- Each permitted subtype must be `final`, `sealed` or `non-sealed` (records are implicitly final)
- The compiler now knows every possible shape – useful for exhaustive `switch`
- Models closed domains: shapes, expressions, payment methods, results

```
public sealed interface Forma
        permits Cerc, Patrat, Dreptunghi { }

public record Cerc(double r) implements Forma { }
public record Patrat(double l) implements Forma { }
public record Dreptunghi(double l, double h)
        implements Forma { }

public record Triunghi(double a, double b, double c)
        implements Forma { }   // compile error: not permitted
```

→ no default case required (compiler knows how many cases ∃)

next slide

Pattern matching for switch (Java 21)

- `case Tip var ->` tests the type and binds a variable
- `when` adds a guard condition
- Over a sealed type the switch is **exhaustive** without `default` – adding a new shape makes this code fail to compile until it is handled
- `case null` handles null explicitly

```
static double arie(Forma f) {
    return switch (f) {
        case Cerc c when c.r() == 0 -> 0;
        case Cerc c       -> Math.PI * c.r() * c.r();
        case Patrat p     -> p.l() * p.l();
        case Dreptunghi d -> d.l() * d.h();
    };          // no default needed
}

static String descrie(Object o) {
    return switch (o) {
        case null      -> "nimic";
        case Integer i -> "intreg " + i;
        case String s  -> "text de " + s.length() + " caractere";
        default        -> "altceva";
    };
}
```

Record patterns: deconstructing data (Java 21)

- A record pattern **extracts the components** directly
- Patterns can be nested
- `var` infers component types; `_` (Java 22) ignores a component

```
static double arie(Forma f) {
    return switch (f) {
        case Cerc(double r)       -> Math.PI * r * r;
        case Patrat(double l)     -> l * l;
        case Dreptunghi(double l, double h) -> l * h;
    };
}

record Segment(Punct a, Punct b) { }

static boolean vertical(Object o) {
    return o instanceof Segment(Punct(var x1, _), Punct(var x2, _))
           && x1 == x2;
}
```

Polymorphism or pattern matching?

OOP (what we did for A1)

Abstract method + overriding

Each subclass implements `arie()` itself.
Best when the set of **types is open** (new subclasses are added often) and the operations are few and stable.

Inheritance + Abstract methods
✓ easy to add a new class → create subclass and implement the abstract method
✗ hard to add a new operation → modify each class.

`data-oriented`

Sealed type + switch

Data types are plain records; operations are `switch`es in one place.
Best when the set of **types is closed** and new operations are added often (compilers, parsers, protocols).

✗ hard to add new class → modify each switch statement bc they must remain exhaustive
✓ easy to add new operations → switch

? Both are type-safe. Modern Java supports both styles – choose per problem, not by habit.

08

Packages, access control and modules

Packages ≠ modular programming

↳ just for organising (folders)

- Group related classes and interfaces
- Manage the **name space**: `java.lang` contains `System`, `Integer`, `String`, …
- Declared with `package name;` – it must be the **first statement** in the file
- The file is stored in a folder with the same name as the package (case-sensitive)
- Subpackages map to subfolders: `structuri.liste` → `structuri/liste/`

structuri/Stiva.java

```
package structuri;
public interface Stiva {
    //...
}
```

structuri/liste/Lista.java

```
package structuri.liste;
public class Lista {
    //...
}
```

Using classes from packages: qualified names

- Without `import`, a class from another package is referred to by its **fully qualified name**: `[pac1.[pac2.[...]]]ClassName`

structuri/ArboreBinar.java

```
package structuri;
public class ArboreBinar {
    //...
}
```

TestStructuri.java

```
public class TestStructuri {
    public static void main(String[] args) {
        structuri.ArboreBinar ab = new structuri.ArboreBinar();
        //...
    }
}
```

The import statement

- One class: `import pac1.[pac2.[...]]ClassName;`
- All classes of a package, **not** its subpackages: `import pac1.[pac2.[...]]*;`
- A file may contain several `import` statements, placed **before** any class declation

→ it does not just concatenate the contents of two files
→ it includes the path of the already-implemented code in my own code

structuri/Heap.java

```
package structuri;
public class Heap {
    //...
}
```

Test.java – without import

```
public class Test {
    public static void main(String[] args) {
        structuri.ArboreBinar ab = new structuri.ArboreBinar();
        structuri.Heap hp = new structuri.Heap();
    }
}
```

The import statement – examples

Test.java – single-class import

```
import structuri.ArboreBinar;

public class Test {
    public static void main(String[] args) {
        ArboreBinar ab = new ArboreBinar();
        structuri.Heap hp = new structuri.Heap();
    }
}
```

Test.java – on-demand import

```
import structuri.*;
import structuri.liste.*;  // needed: * excludes
                           // subpackages
public class Test {
    public static void main(String[] args) {
        ArboreBinar ab = new ArboreBinar();
        Heap hp = new Heap();
        Lista li = new Lista();
    }
}
```

package + import

- The `package` statement comes **before** any `import`
- `java.lang` is imported implicitly by the compiler

Order in a source file: `package` → `import`s → type declarations

algoritmi/Backtracking.java

```
package algoritmi;
import structuri.*;

public class Backtracking {
    //...
}
```

The unnamed (default) package

```java
// Persoana.java
public class Persoana { /*...*/ }

// Complex.java
class Complex { /*...*/ }

// Test.java
public class Test {
    public static void main(String[] args) {
        Persoana p = new Persoana();
        Complex c = new Complex();
        //...
    }
}
```

A `.java` file without a `package` statement belongs to the **unnamed package**. Fine for small tests; real projects always use named packages.

Name collisions

- Two imported packages may contain classes with the same name: use the **qualified name**

unu/A.java

```
package unu;
public class A {
    //...
}
```

doi/A.java

```
package doi;
public class A {
    //...
}
```

Test.java

```
import unu.*;
import doi.*;
public class Test {
    public static void main(String[] args) {
        A a = new A();          // compile error: ambiguous
        unu.A a1 = new unu.A();
        doi.A a2 = new doi.A();
    }
}
```

Access modifiers for class members

Modifier	Same class	Same package	Subclass (other package)	Everywhere
`public`	✓	✓	✓	✓
`protected`	✓	✓	✓	–
`(none) – package-private`	✓	✓	–	–
`private`	✓	–	–	–



Top-level classes and interfaces can only be `public` or package-private (no modifier). `private` and `protected` are allowed only for nested classes.

Access modifiers – example

- `Test` is in the unnamed package and imports `structuri.*`. Which statements compile?

structuri/Nod.java

```
package structuri;
class Nod {
    private Nod urm;
    public Nod getUrm() {...}
    void setUrm(Nod p) {...}
}
```

structuri/Coada.java

```
package structuri;
public class Coada {
    Nod cap;
    public Coada() { cap = null; }
    Coada(int i) {...}
}
```

Test.java

```
import structuri.*;
class Test {
    void main() {
      Coada c = new Coada();
      Nod n = new Nod();
      Coada c2 = new Coada(2);
    }
}
```

? `new Coada()` – OK (public). `new Nod()` – error: class `Nod` is not public. `new Coada(2)` – error: constructor is package-private.

protected vs. package-private across packages

unu/A.java

```
package unu;
public class A {
    A(int c, int d) {...}
    protected A(int c) {...}
    public A() {...}
    protected void f() {...}
    void h() {...}
}
```

unu/DA.java

```
package unu;
class DA extends A {
    DA(int c) {
        super(c);
    }
}
```

doi/DDA.java

```
package doi;
import unu.*;
class DDA extends A {
  DDA(int c) {
    super(c);        // ?
  }
  DDA(int c, int d) {
    super(c, d);     // ?
  }
  protected void f() {
    super.h();       // ?
  }
}
```

private members are not accessible in subclasses

Persoana.java

```java
public class Persoana {
    private String nume;
    private int varsta;
    public Persoana(String nume, int varsta) {
        this.nume = nume;
        this.varsta = varsta;
    }
}
```

Angajat.java

```java
public class Angajat extends Persoana {
    private String departament;
    public Angajat(String nume, int varsta, String dep) {
        super(nume, varsta);
        departament = dep;
    }
    public String descriere() {
        return nume + ", " + varsta;  // ERROR: private
    }
}
```

! `nume` and `varsta` exist in every `Angajat` object, but they are `private` in `Persoana`: the subclass cannot use them by name.

protected members

Persoana.java

```
public class Persoana {
    protected String nume;
    protected int varsta;
    public Persoana(String nume, int varsta) {
        this.nume = nume;
        this.varsta = varsta;
    }
}
```

Angajat.java

```
public class Angajat extends Persoana {
    protected String departament;
    public Angajat(String nume, int varsta, String dep) {
        super(nume, varsta);
        departament = dep;
    }
    public String descriere() {
        return nume + ", " + varsta;  // OK: protected
    }
}
```

i `protected` members are visible in the class, in its subclasses and in the same package. Alternative: keep fields `private` and expose `protected` or `public` getters.

Modules (Java 9): a brief look

- A **module** groups packages and declares, in `module-info.java`:
  - which modules it **requires**
  - which packages it **exports** – all others are hidden, even if `public`
- The JDK itself is split into modules (`java.base`, `java.sql`, …)
- Java 25: `import module java.base;` imports all packages exported by a module
- Small projects and this course can ignore modules (classpath mode)

module-info.java

```
module ro.ubb.catalog {
    requires java.sql;
    exports ro.ubb.catalog.api;
    // ro.ubb.catalog.internal stays hidden
}
```

Java 25 – module import

```
import module java.base;   // List, Map, Path, ...

public class Demo {
    public static void main(String[] args) {
        List<String> l = List.of("a", "b");
        System.out.println(l);
    }
}
```

Summary

- Reuse code by **composition** (has-a) or **inheritance** (is-a); override with `@Override`, reach the parent with `super`
- Late binding selects the method from the object's class – the basis of polymorphism
- Abstract classes share state and code; interfaces define types and, since Java 8, can carry default methods
- Override `equals` **and** `hashCode` together – or use a **record**
- Enums for fixed sets of constants; sealed types + pattern matching `switch` for closed hierarchies
- Packages organize code; access modifiers and modules control visibility

Questions?

Next: Lecture 3 – Exceptions and Generics